



Canyon Carnage

Soutenance finale



Team

- Arthur Bru
- Gauthier Bouyjou
- Henri Tran
- Jérôme Hussenet
- Nael Bellon





Pitch & Intention



Univers et ambiance

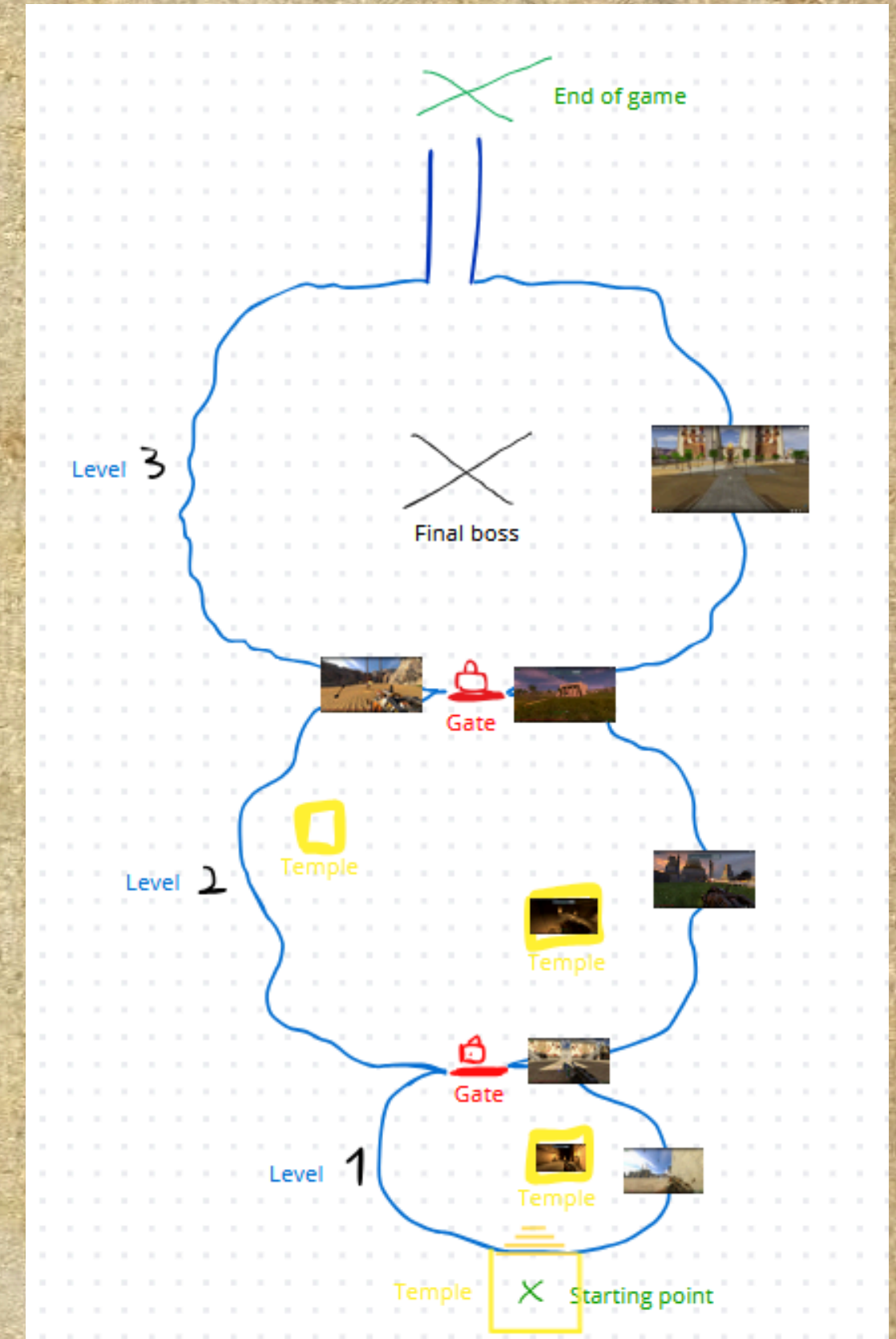


- Un canyon colossal (semi-openworld).
- Exploration de temple avec des couloirs sombres et piégés.
- Arènes ouvertes envahies de créatures et de boss.



Objectifs du joueur

- S'échapper du Canyon
- Exploration monde semi-ouvert



Objectifs du joueur



Objectifs du joueur



Combattre des ennemies



Objectifs du joueur

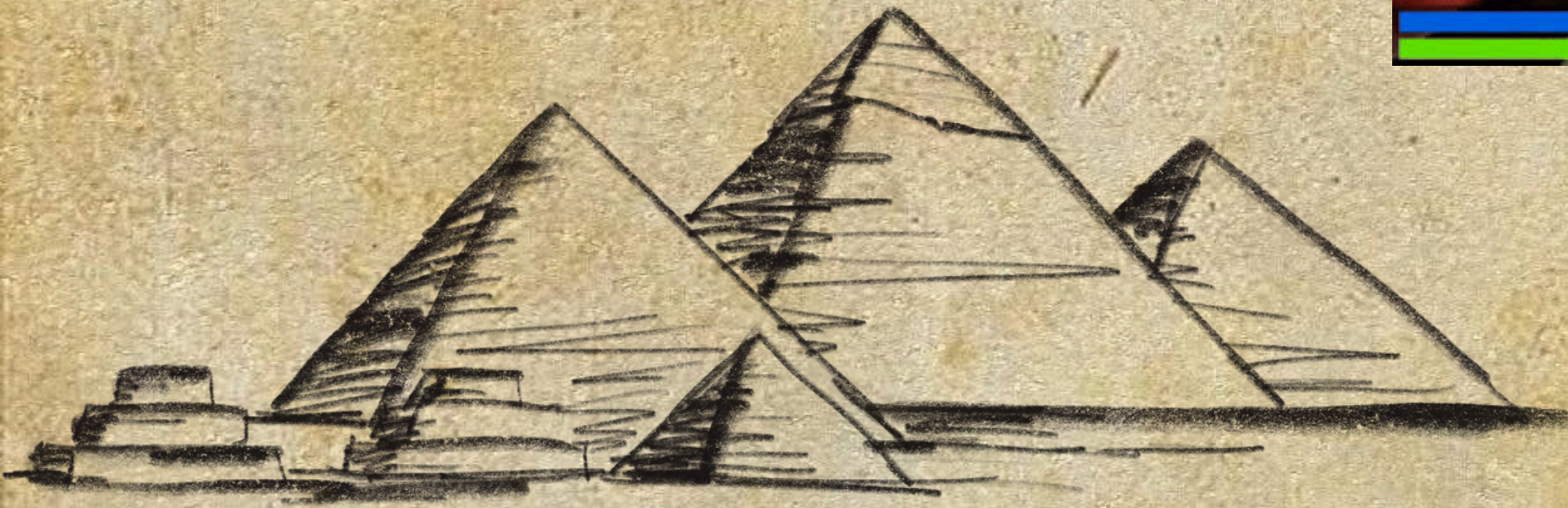


Résolution d'énigmes



Intention de conception

- Gameplay dynamique et nerveux (doom like)
- Combat contre de grandes vagues d'ennemis
- Stress constant du joueur (munitions limitées)
- Simplicité des systèmes (armes)
- Espaces vastes (arènes)

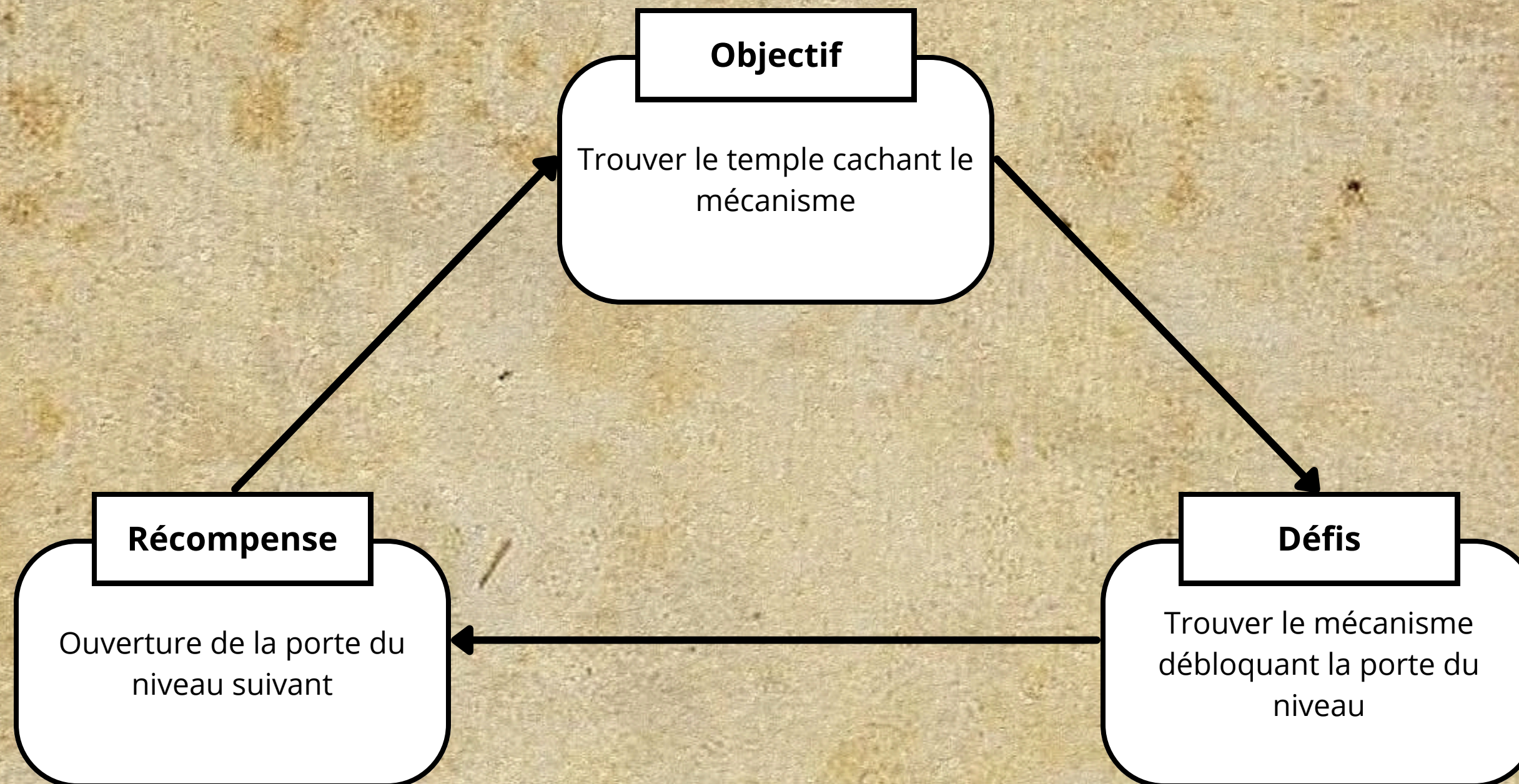




Game design & Expérience joueur

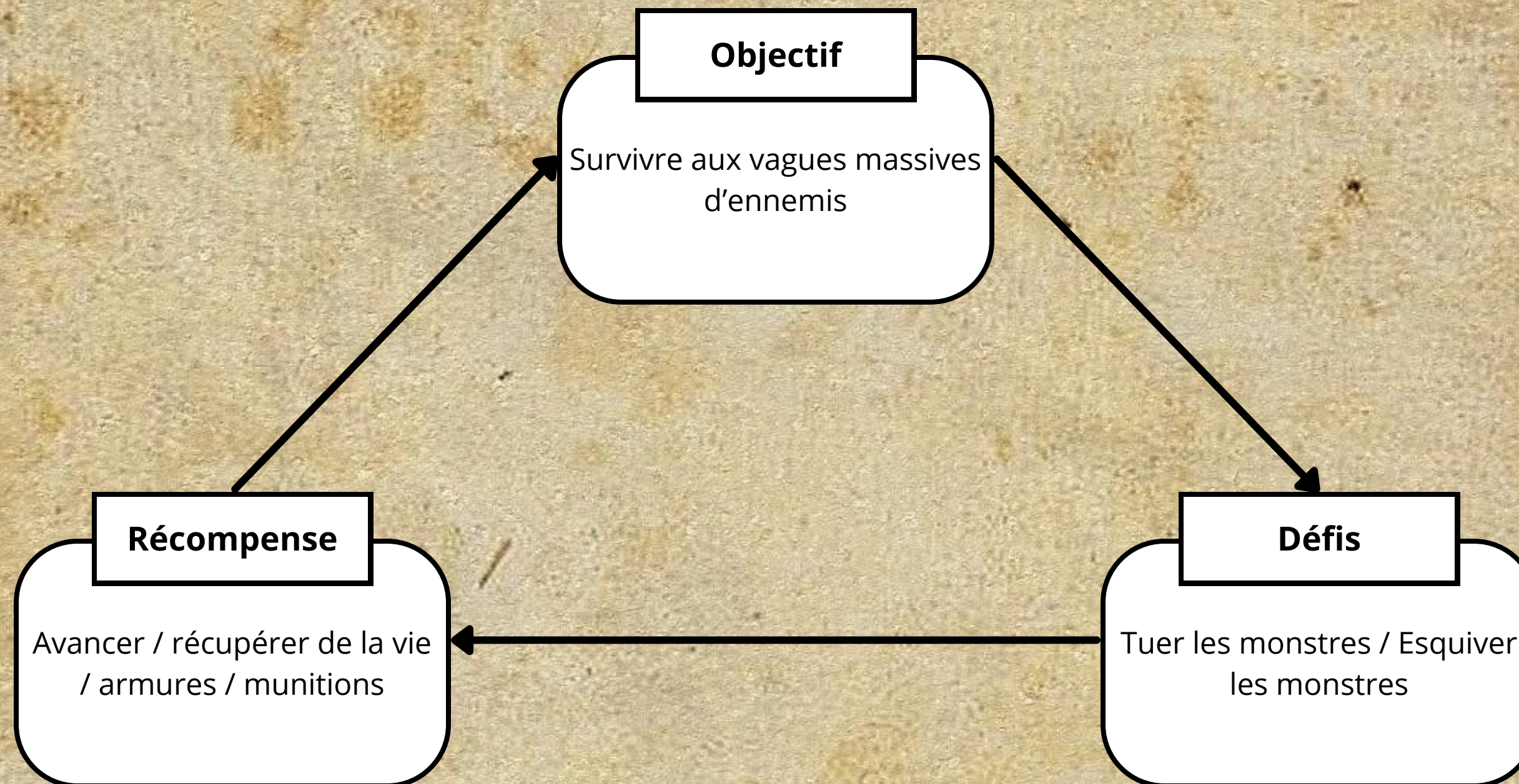
Boucle de gameplay

High Loop : Boucle de niveau

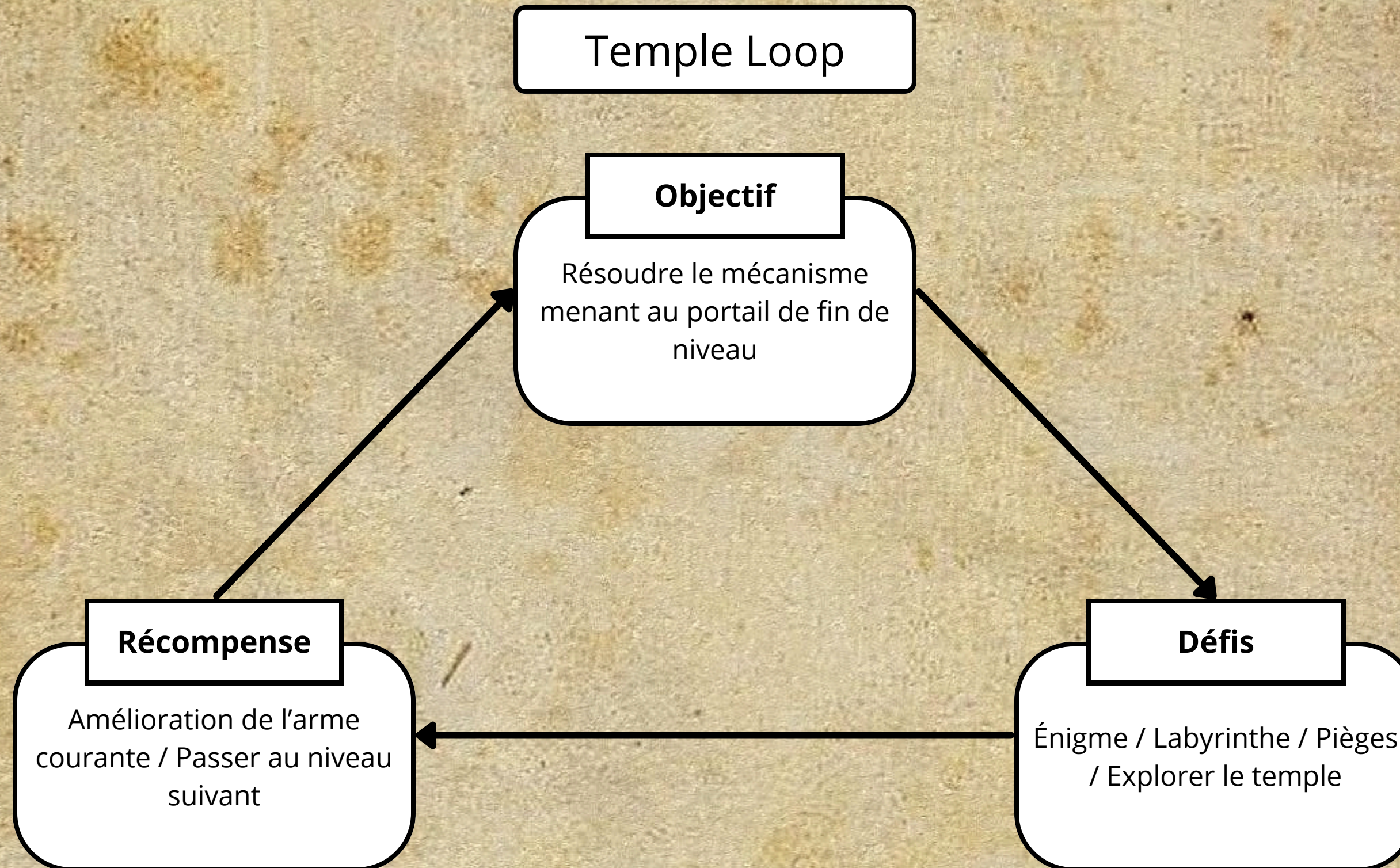


Boucle de gameplay

Core Loop : Boucle de combat



Boucle de gameplay



Mécaniques principales

Système de combat



Roue des armes



Pièges



Puzzles



Gestion du rythme



Gestion du rythme



Progression du joueur



Indication des armes

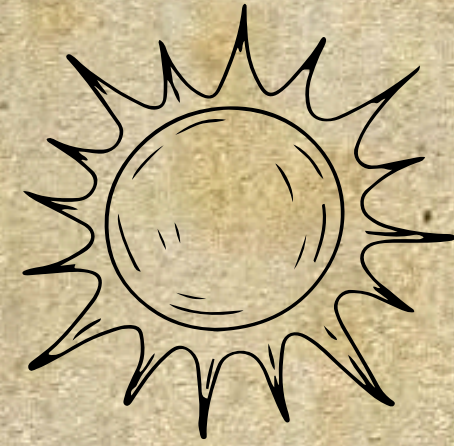
Progression du joueur



Énigme et passage au niveau suivant avec
cinématique

Feedbacks visuels et sonores (gamefeel)





Demo

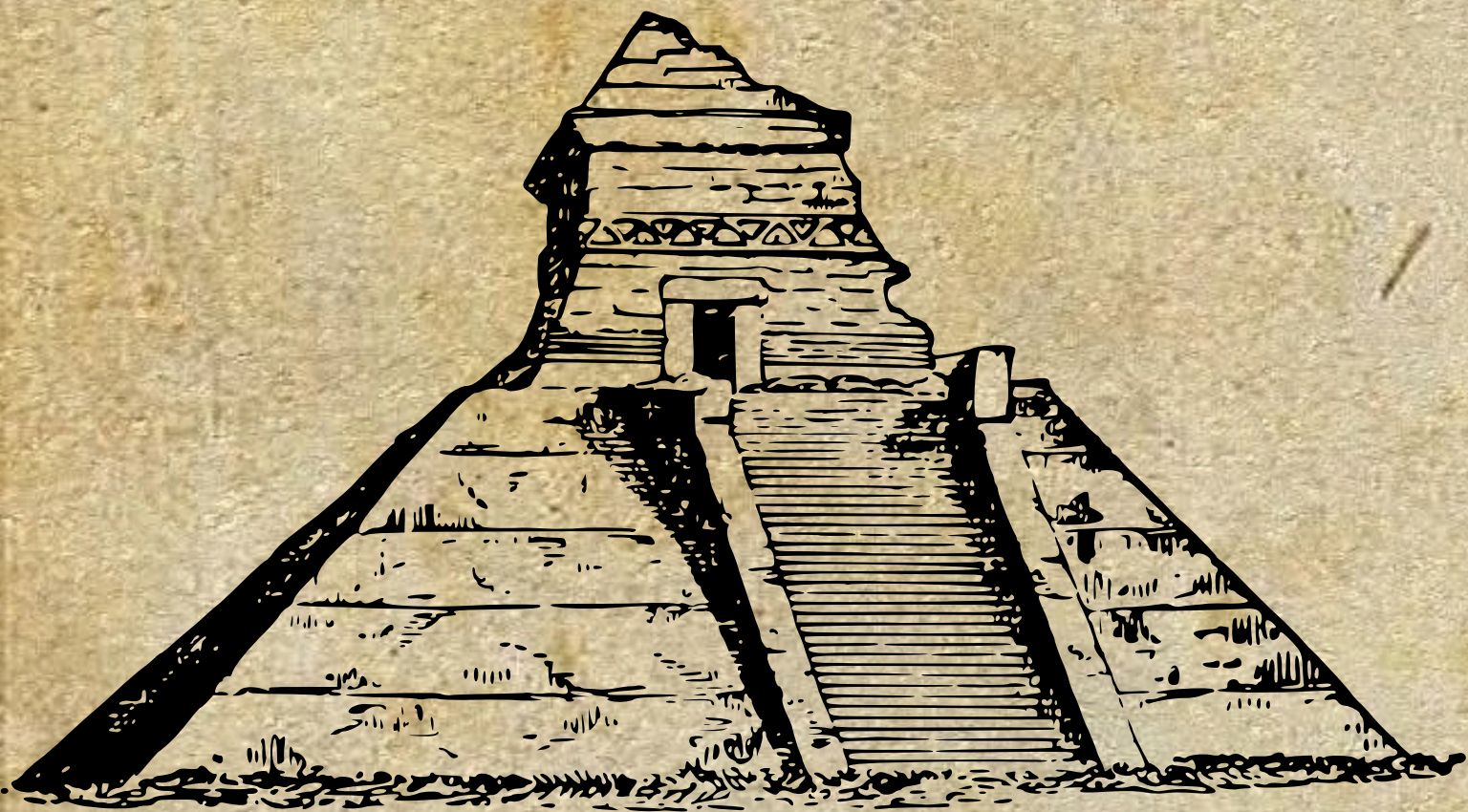




MAINTENIR ESPACE POUR PASSER



Technique et architecture



Organisation du projet



Arborescence du projet

Content/

└ CanyonCarnage/

└└ Maps/

└└ Core/

└└ Characters/

└└ Enemies/

└└ Weapons/

└└ Art/

└└ Audio/

└└ UI/

└└ FX/

└└ AI/

└ Developer/

Plugins/

 Canyon Carnage **APP** 21/06/2026 23:31
Une Pull Request vient d'être mergée dans **Develop** ! N'oubliez pas de pull la Develop pour être à jour !

PoloBongo
#191 add end game ui
Aucune description fournie.

Branche source	Branche cible	Validée par
add/191-endgame-ui	Develop	PoloBongo

 Pull Request GitHub • 21/06/2026 23:30

22 juin 2026

 Canyon Carnage **APP** 22/06/2026 02:07
Une Pull Request vient d'être mergée dans **Develop** ! N'oubliez pas de pull la Develop pour être à jour !

46Yuu
Temple2 merge
added temple 2 map and modified temple 1, remember to get the assets from winscp

Branche source	Branche cible	Validée par
Temple2	Develop	46Yuu

 Pull Request GitHub • 22/06/2026 02:07

 Canyon Carnage **APP** 22/06/2026 16:01
Une Pull Request vient d'être mergée dans **Develop** ! N'oubliez pas de pull la Develop pour être à jour !

StratiX0
[MERGE] TrapsImprov to Develop
Aucune description fournie.

Branche source	Branche cible	Validée par
TrapsImprov	Develop	StratiX0

 Pull Request GitHub • 22/06/2026 16:01

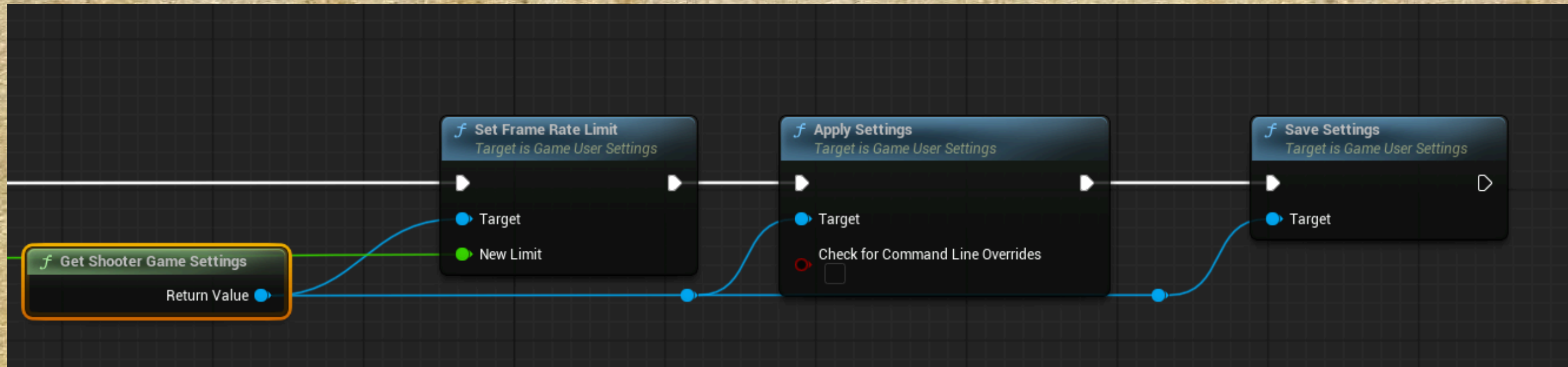
Conventions de nommage

Bonnes pratiques de nommage et de structure

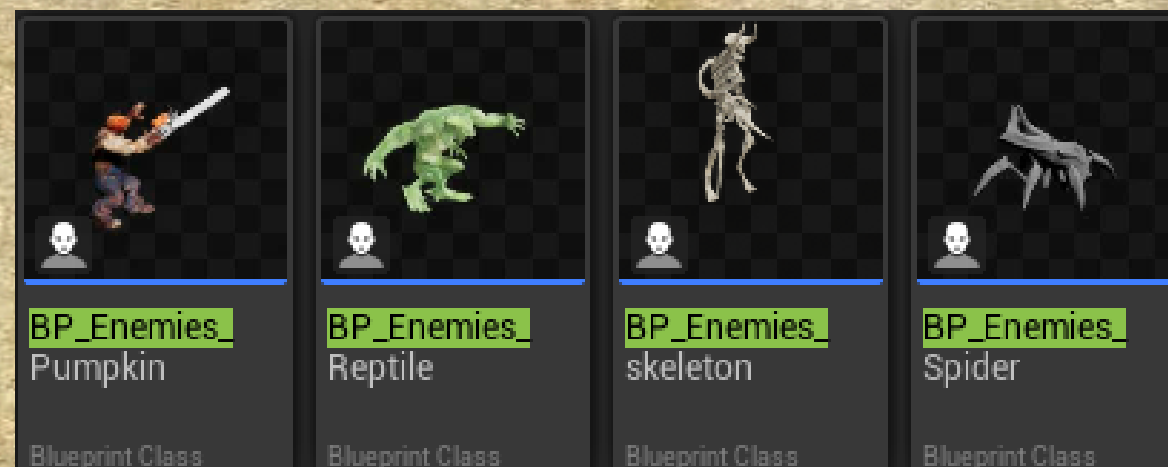
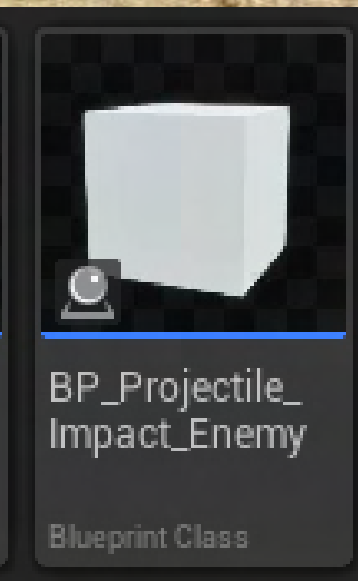
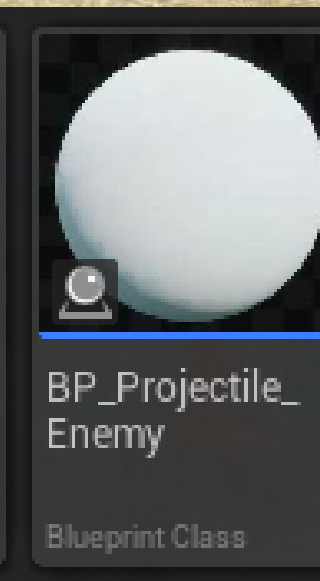
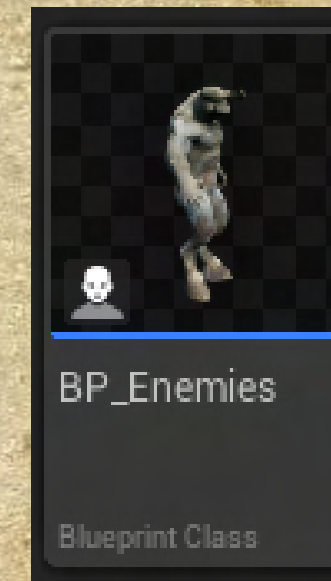
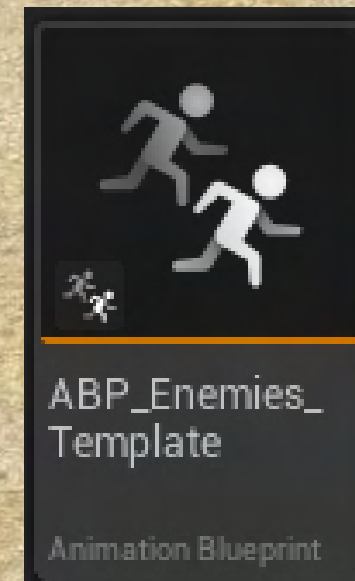
- Convention de préfixe Epic: BP_, SM_, SK_, T_, M_, S_ ou CUE_. Exemple: BP_PlayerCharacter, SM_RockLarge, T_UI_HealthBar. [Conventions](#)
- PascalCase sans espaces ni caractères spéciaux: MyLevel, RocketPistol. Évite les soucis de scripts, CI et contrôle de source. [Réf](#) • [Détails](#)
- Dossier projet unique: tout sous Content/CanyonCarnage/. Ne rien laisser dans Content/ ni créer un dossier "Assets". [Pourquoi](#)
- Maps centralisées: toutes les maps sous Content/CanyonCarnage/Maps/ pour simplifier l'ouverture et la cuisson. [Réf](#)
- Core pour les fondations: GameMode, PlayerController, Character principal, GameState, PlayerState. Éviter les modifications directes par les designers. [Réf](#)
- Éviter les dossiers redondants globaux (Meshes, Textures, Materials). Utiliser les filtres du Content Browser et les préfixes. [Explications](#) • [Static Mesh filter](#) • [Pourquoi pas "Assets"](#)
- Clarté et faible profondeur: quelques niveaux max, dossiers par fonctionnalité claire (UI, Audio, AI...). [Synthèse](#)

Architecture Blueprint

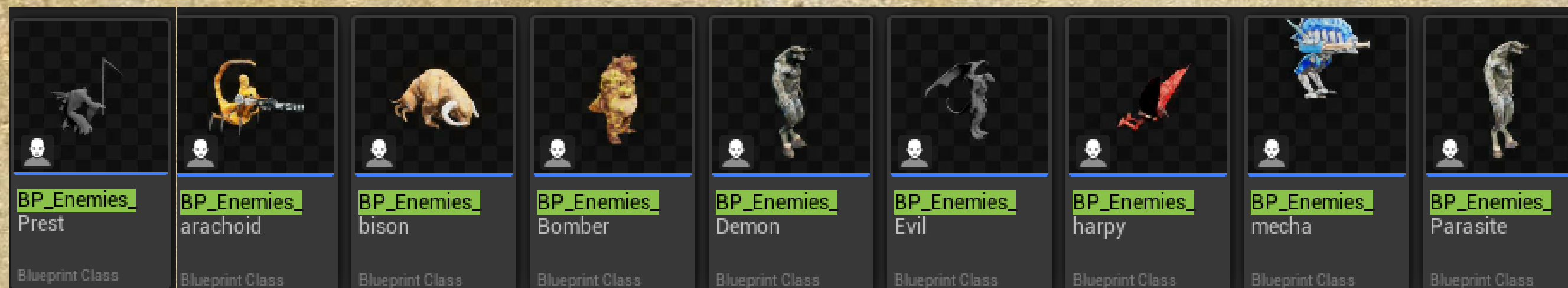
Facile d'accès



IA & Modularité



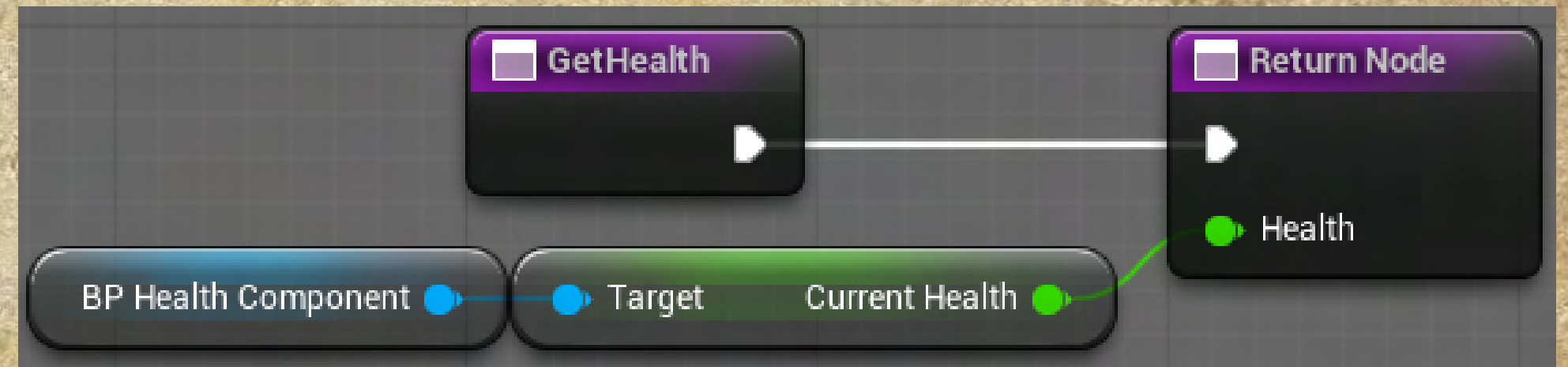
Bluperint generique pour tous les monstres



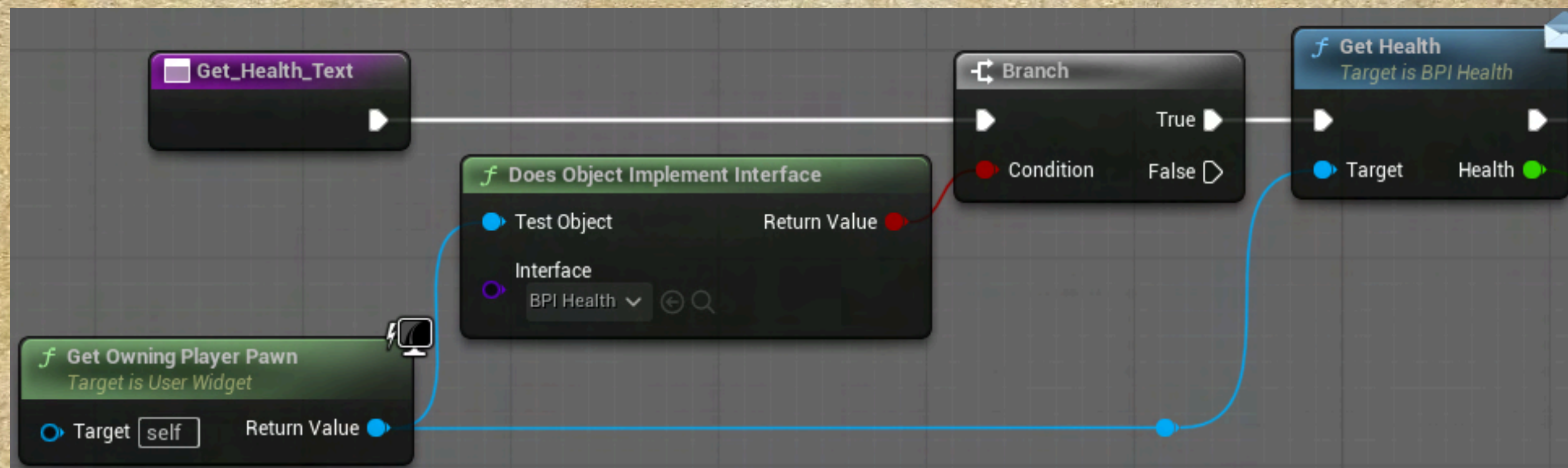
Interfaces



Événement “OnHeal” de l’interface dans le Player Character



Lien entre “GetHealth” de l’interface avec la vie du Player Character



Récupérer la vie pour le HUD

Optimisations réalisées

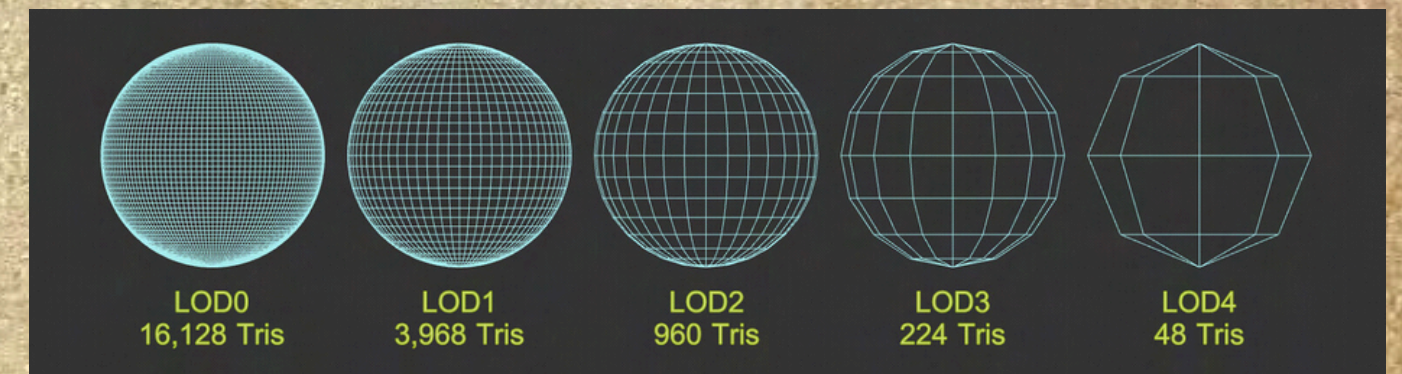
- Virtual Shadow Maps → Shadow Maps
- Lumen + désactivation du Ray Tracing Hardware
- DirectX 11 envisagé
- Limiter la taille maximale des textures



Nanite Désactivé

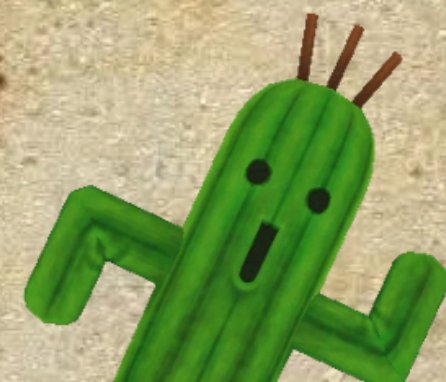
Résultat :

- Performances doublées
- Ex : Niveau Jungle ~35 FPS → ~100 FPS



Création de LODs pour les modèles

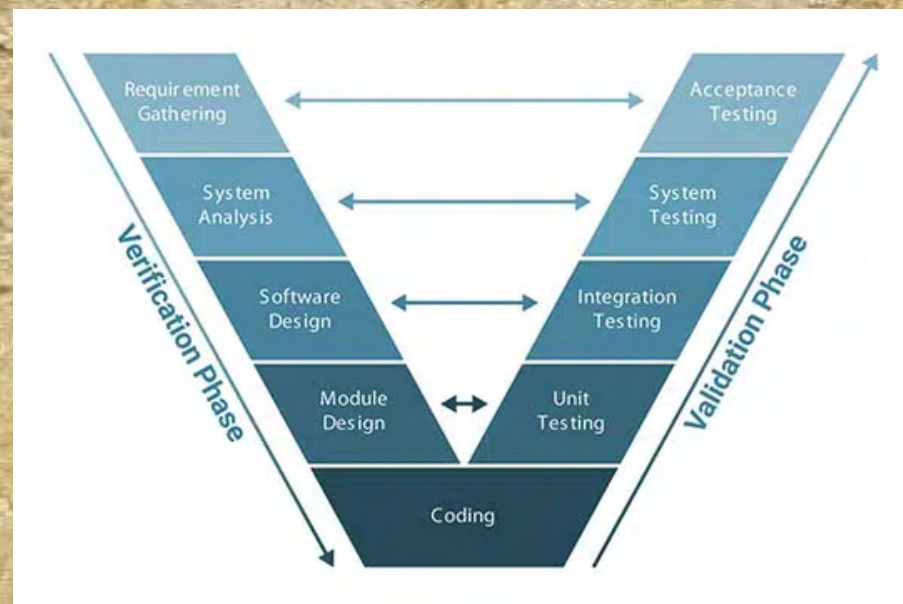
Production et méthodologie



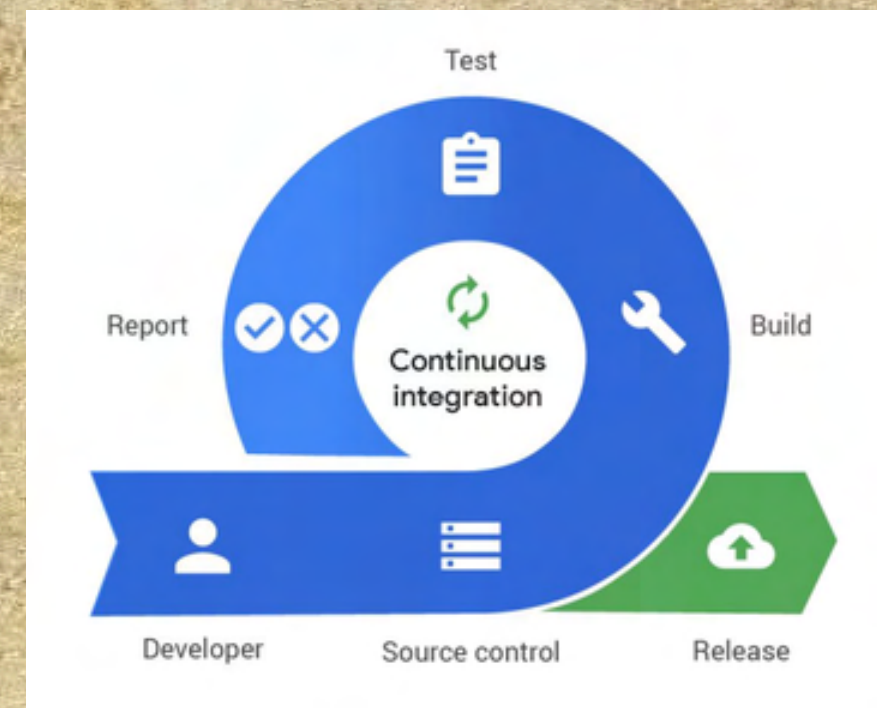
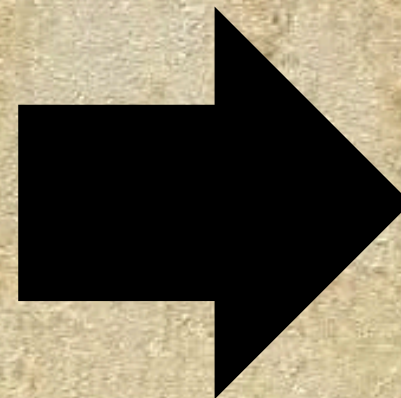
Répartition des rôles

	C++	Gameplay développeur	IA & vagues	Optimisations	Level design, lighting	Énigmes / Puzzles	Animations	UI
Gauthier Bouyjou		✓	✓	✓	✓		✓ Rigging (blender)	
Jérôme Hussenet	✓	✓		✓		✓	✓	✓
Henri Tran	✓	✓			✓	✓	✓	✓
Arthur Bru	✓	✓		✓	✓			✓

Organisation du travail



Cycle en V



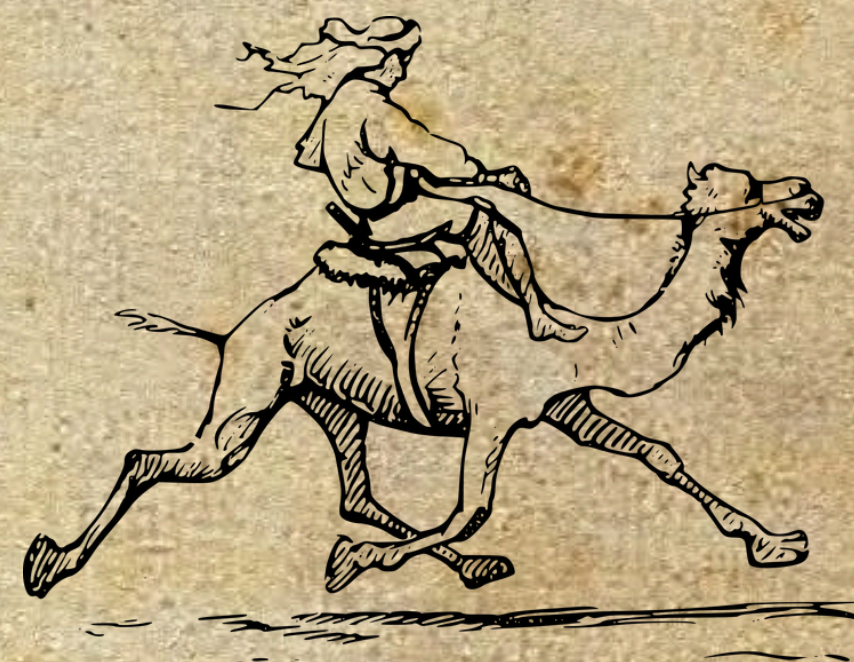
Agile

Difficultés rencontrées

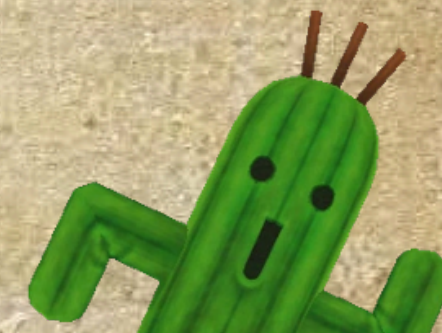
- Conflit entre fichiers Unreal à cause des .uasset
- Diversité de performance (PC différents)
- Perte d'un membre dans l'équipe

	Mineure	Significative	Sévère
Certain	Assets		Bugs
Probable	BP UE5	Performance	
Rare	Scope		Abandon

Risque	Description	Plan d'atténuation
Assets (modélisation)	Charge de travail liée à la création in-house des assets, niveaux, gameplay et HUD (Graphisme/Design)	Définir une direction artistique simple et minimaliste. Utiliser des outils de création de contenu rapides (Blender).
Conflit fichier UE5 binaire	Impossibilité de gérer les conflits sur fichier binaires avec git	Séparer le code avec des composants ou des packages de fonction blueprint.
Performance	Problèmes de performance 3D (optimisation)	Intégrer des tests de performance et d'optimisation dès le début du développement (profiling)
Scope	Scope Creep (ajout de fonctionnalités non planifiées)	Définir et faire valider le Scope de manière rigide (PBS). Utiliser des itérations courtes (Agile).
Abandon	Manque de disponibilité d'un membre de l'équipe	Documenter le code et les processus. Assurer la redondance des compétences techniques.
Jeu buggé	Bugs critiques découverts tardivement.	Mettre en place des phases de tests régulières et des revues de code.



Direction artistique et immersion



Direction artistique

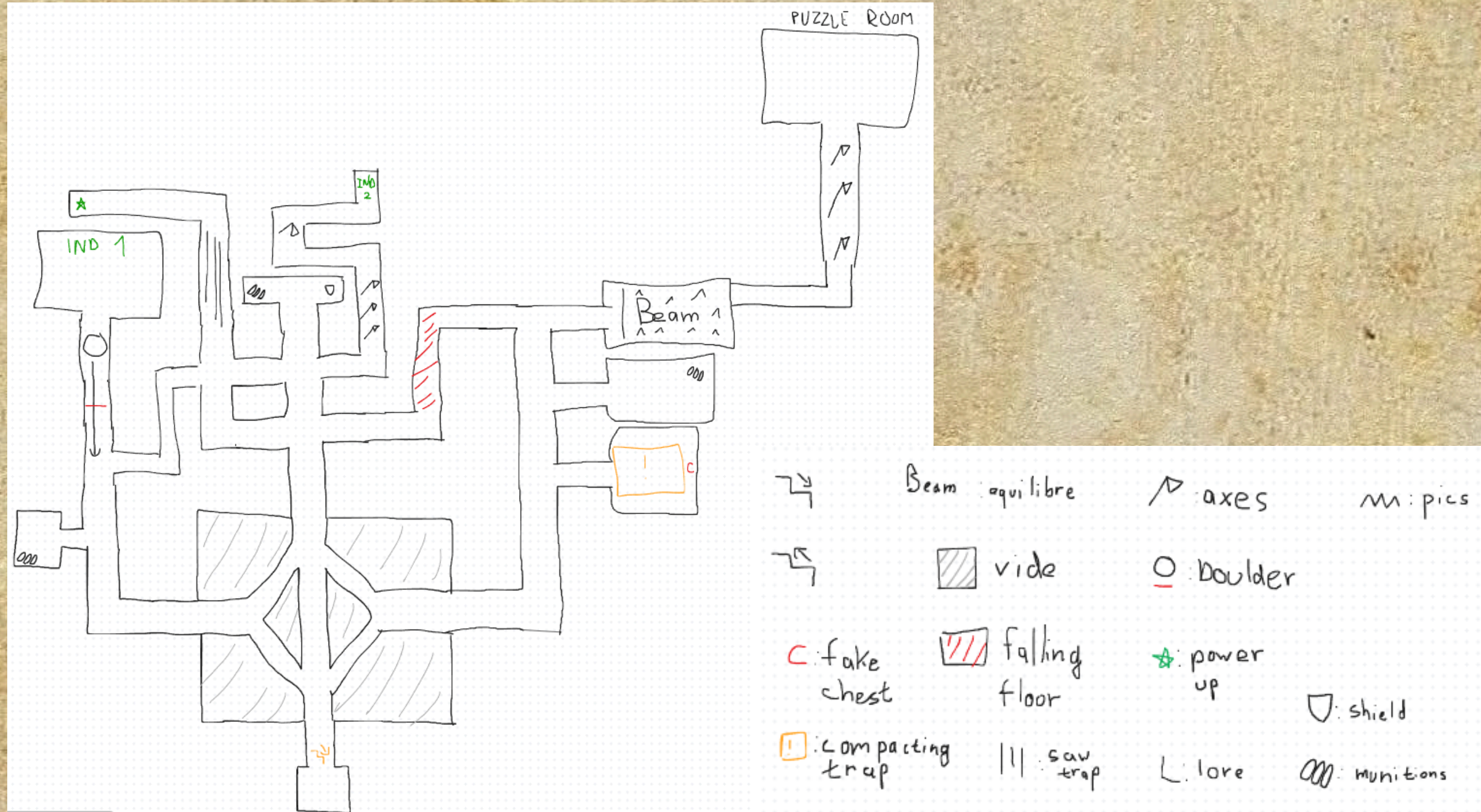
- Map en ligne droite, dans un canyon.
- Ennemis de style varié
- Environnement désert/jungle
- Temples style Maya/Égyptiens
- Sous-sol des temples : faible luminosité, plein de piliers



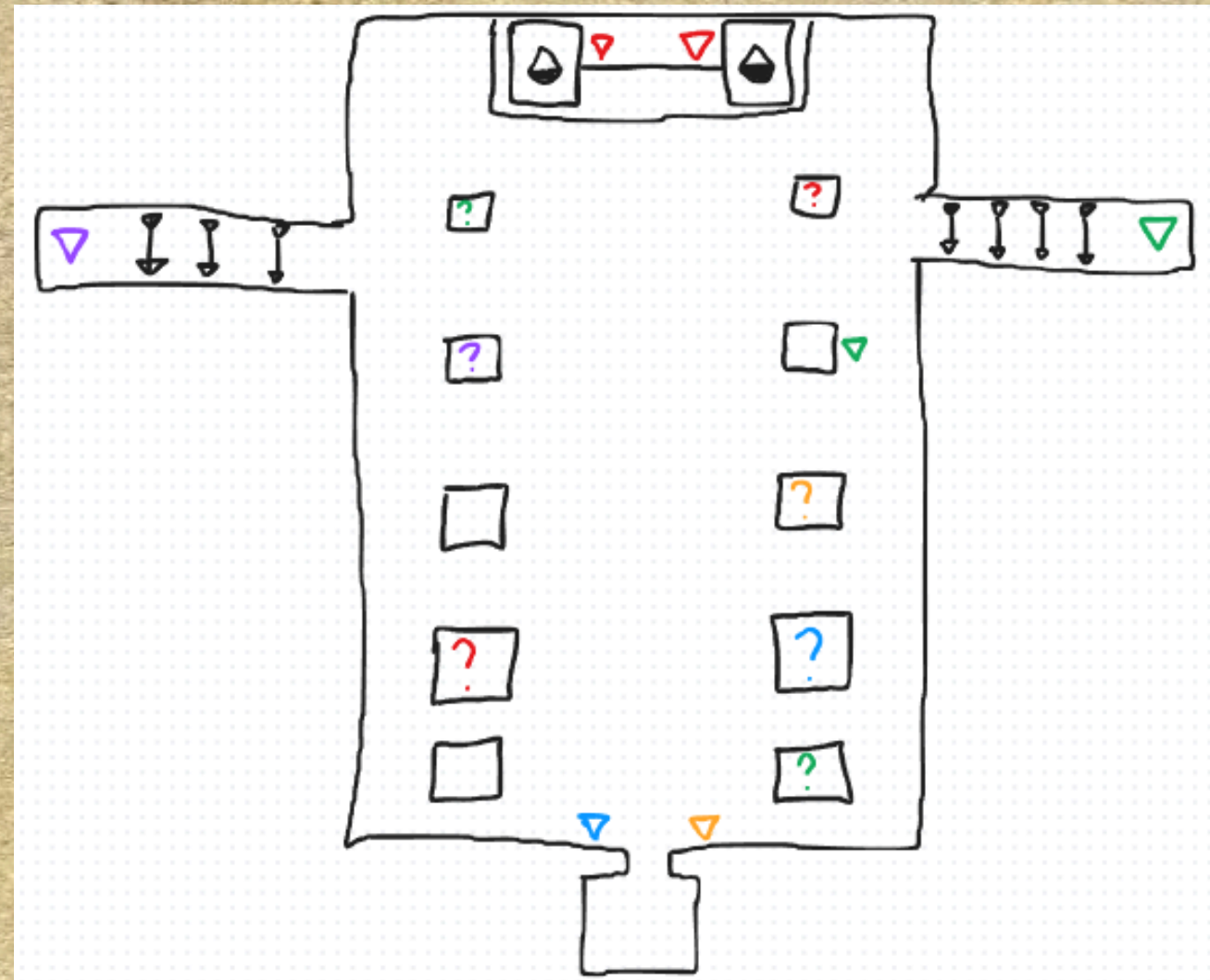
Direction artistique



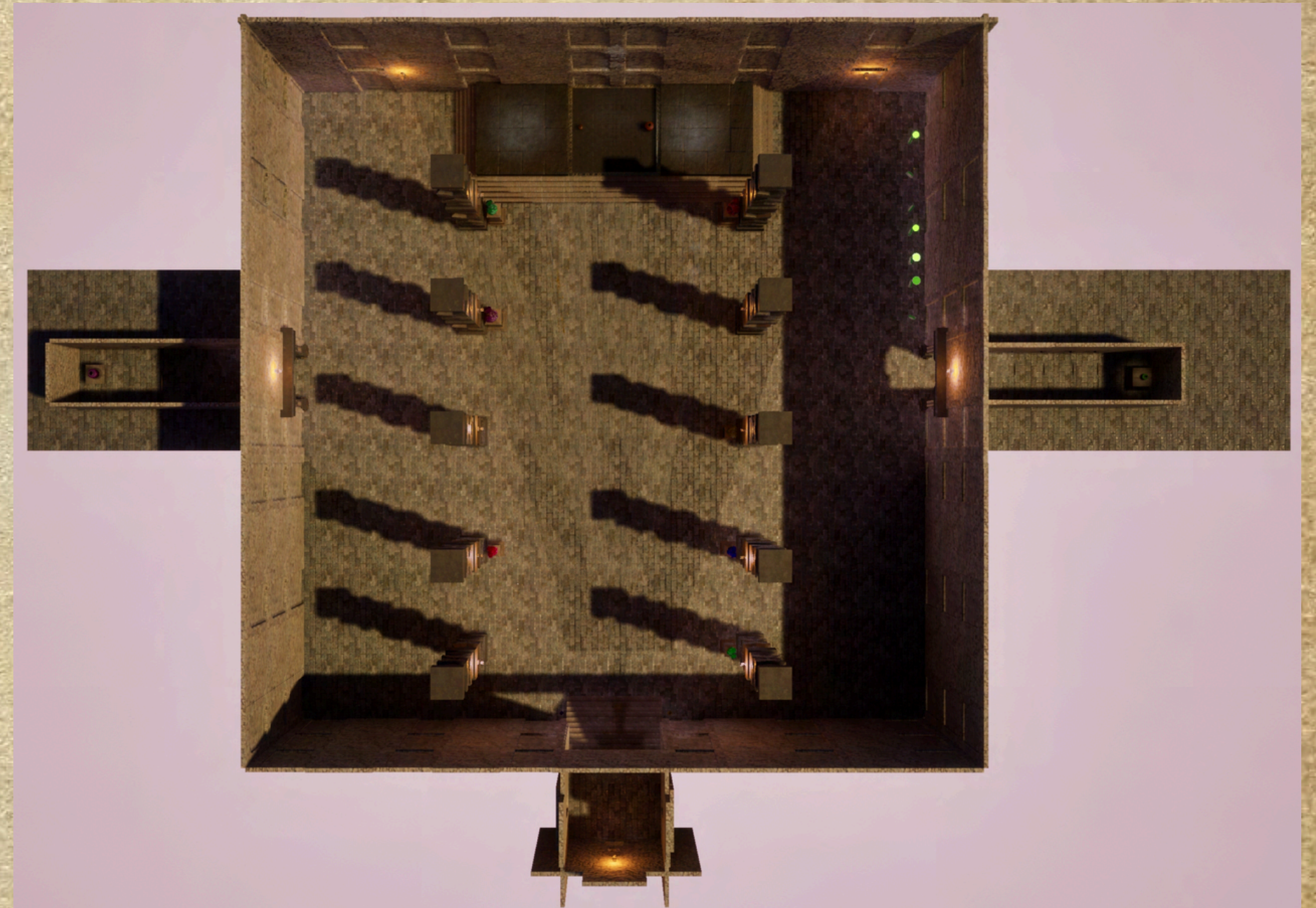
Level Design



Level Design



? = hints
↓ = Arrow Trap
▽ = Weight object
□—□ = scales / weighing machine



Shaders



Barre de Vie / Shield



Combo



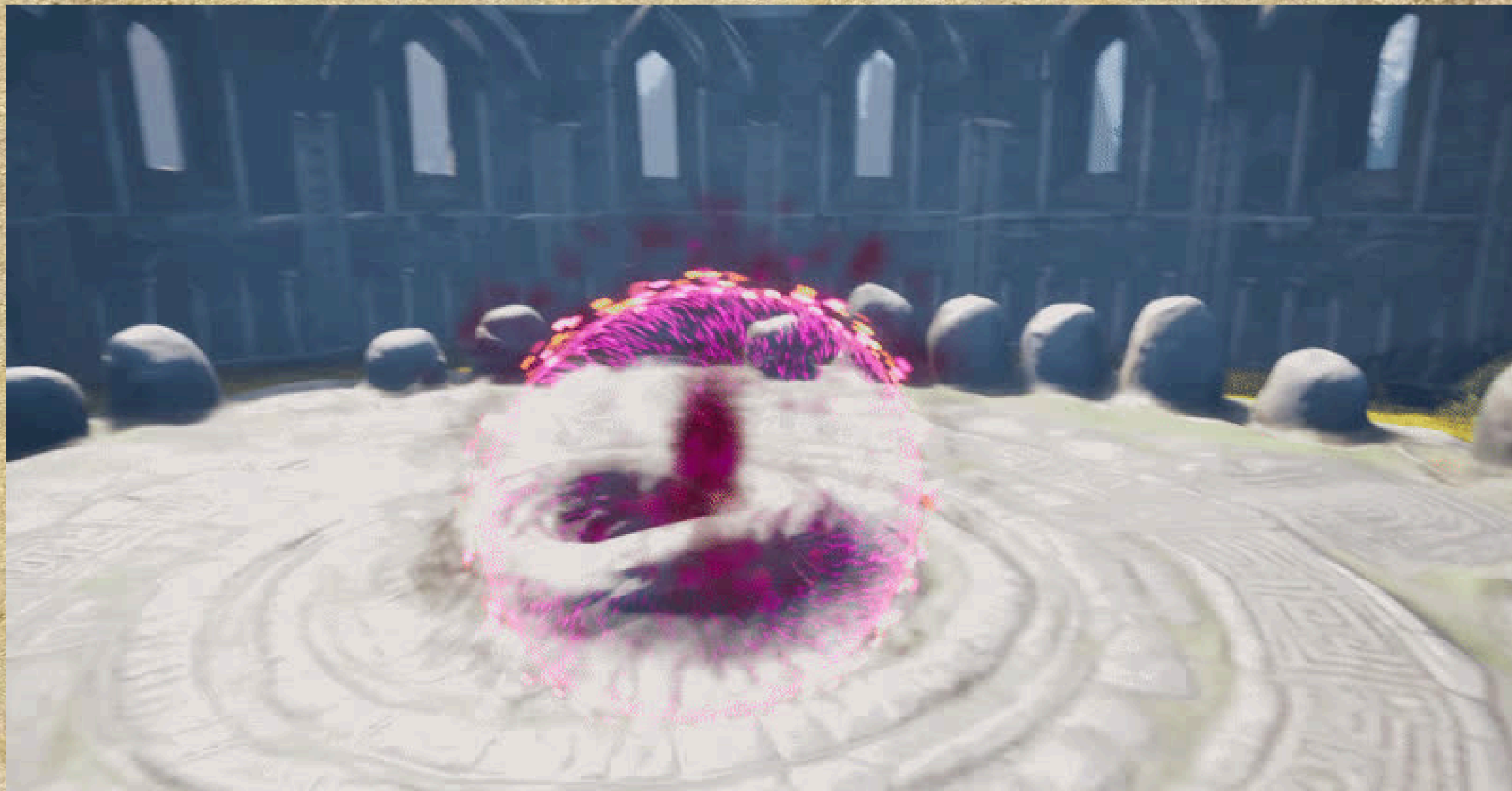
Sang (éclaboussure et tâches)

Shaders



Textures selon les caractéristiques du terrain
Génération de l'environnement (herbes, arbres, rochers)

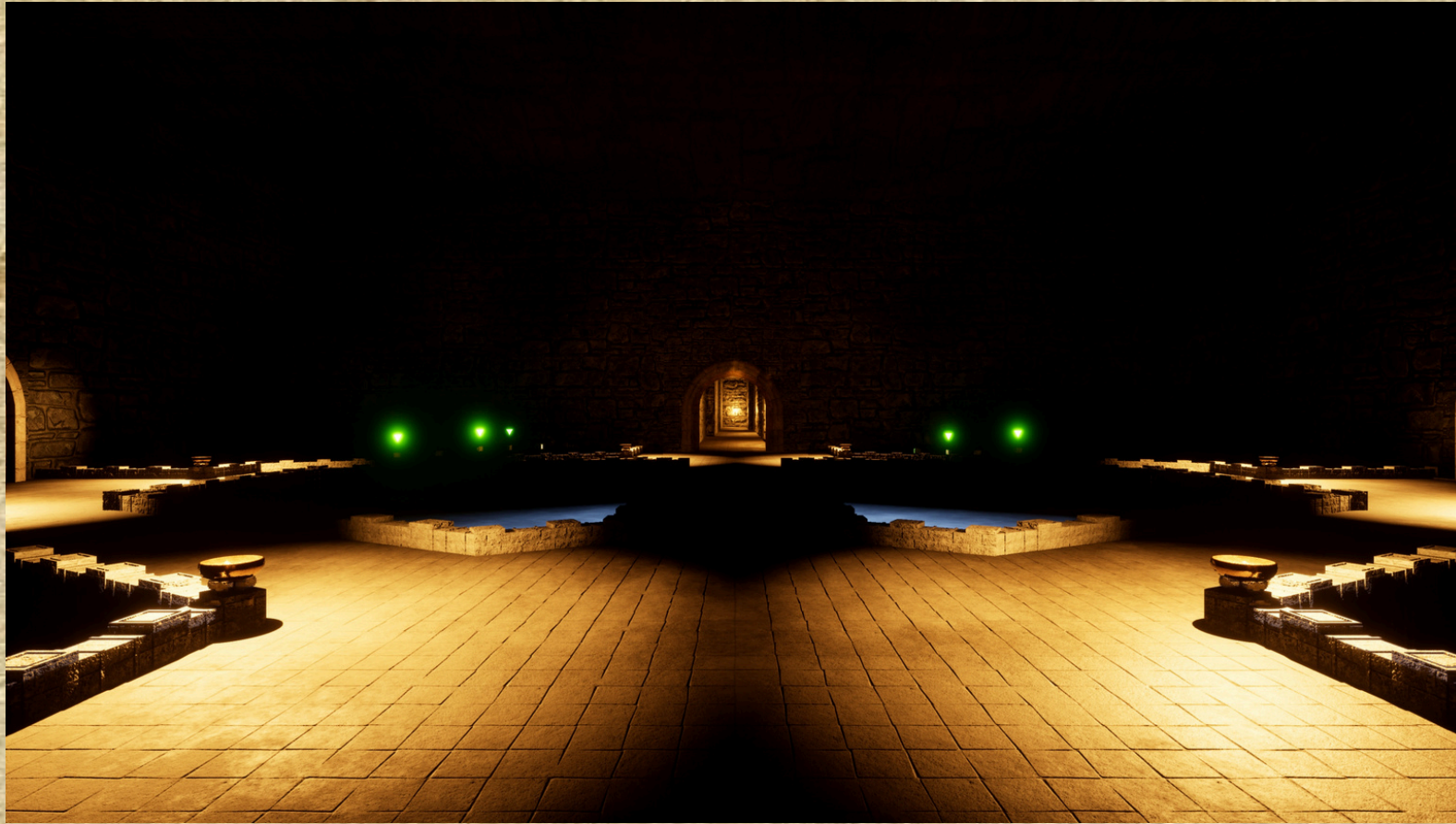
VFX



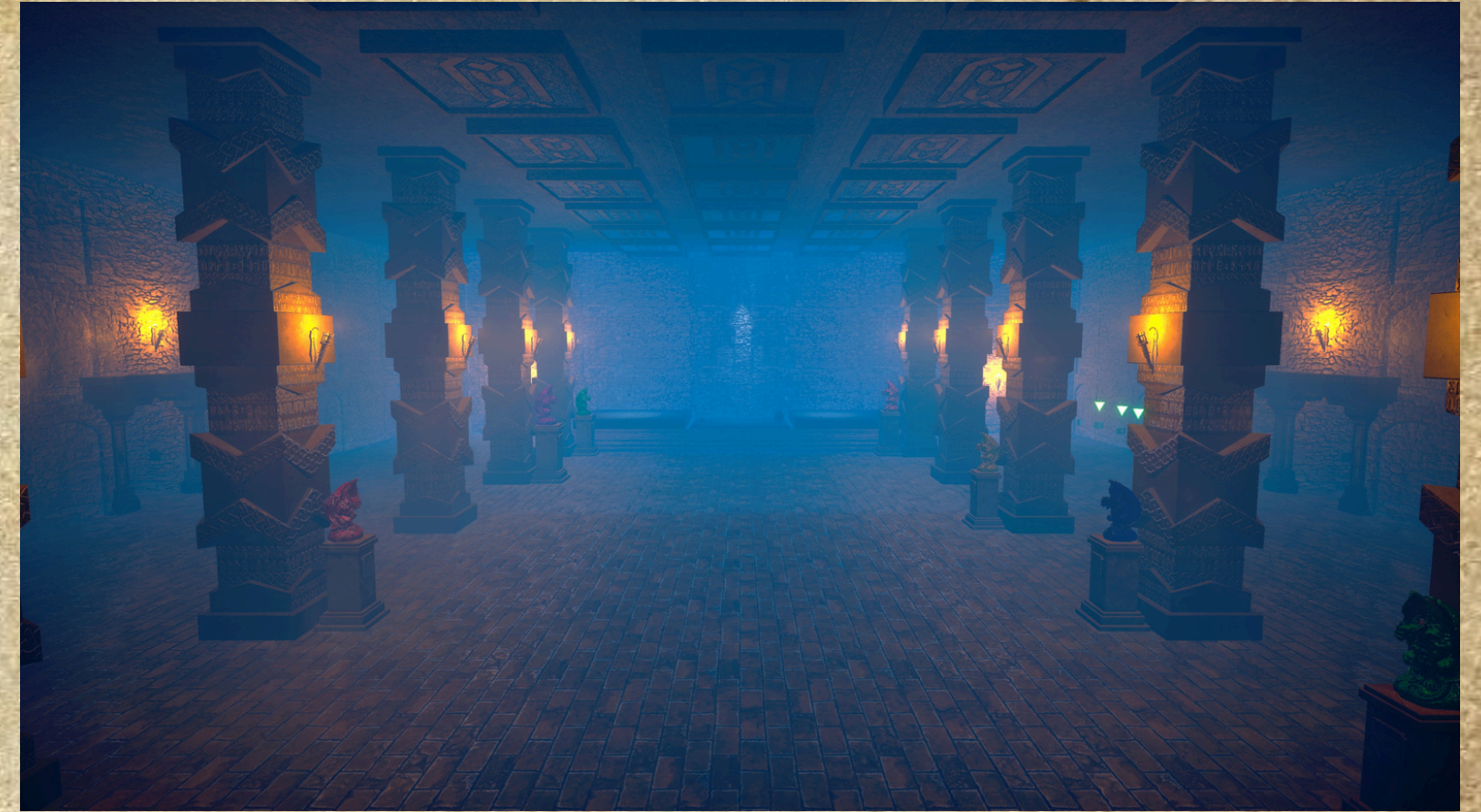
VFX



Éclairage



Temple 1



Temple 2



Ambiance sonore

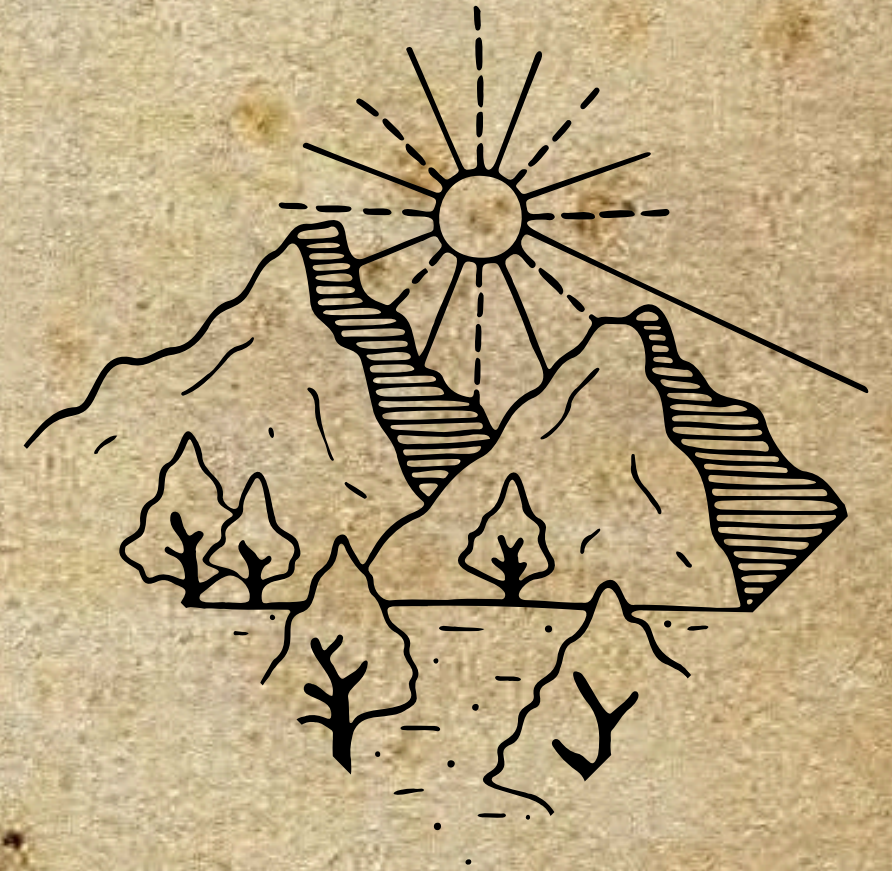


Spatialisation audio

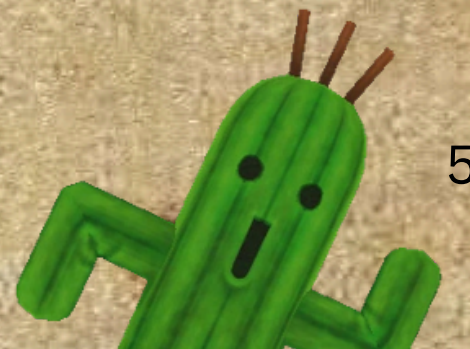
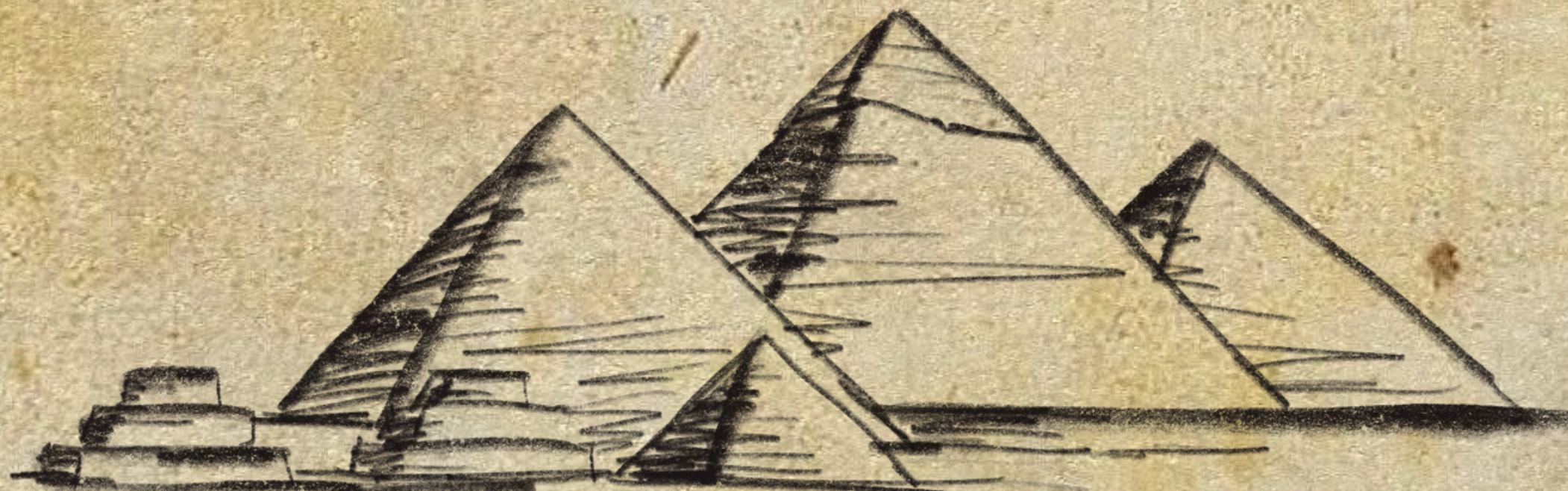
Environnement
(oiseaux, orage, corbeaux,
porte qui s'ouvre, flame)

Déplacement des monstres,
projectiles,
spawn et début vague





Conclusion et post-mortem



Les réussites du projet

- Agilité face au départ d'un membre
- Respect du cahier des charges
- Collaboration fluide, feedback intégré



Les points d'amélioration



- Optimisation globale des performances
- Éclairage des temples
- Équilibrage de la difficulté (dégâts reçus)
- Ajout de tests de non régression

Les enseignements tirés

- Communication bienveillante (aider > blesser)
- Itération par playtests
- Priorisation tâches principales / secondaires
- Repo Git léger = workflow agréable



Ce qu'on ferait différemment



- Agile dès le départ (flexibilité face aux imprévus)
- Playtests réguliers (build auto si budget)
- Estimations de tâches plus réalistes

Questions and Discussion

